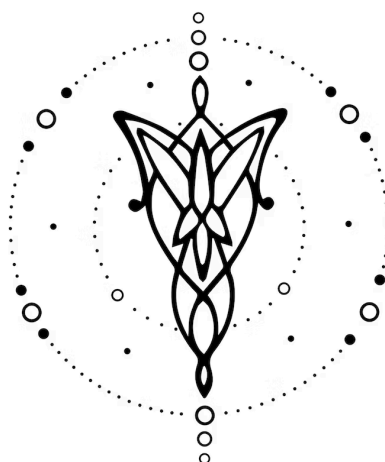




Especificación - RPlang

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Dominio	1
3. Construcciones	2
4. Casos de Prueba	2
5. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Santiago	Bassi	64.643	sabassi@itba.edu.ar
Ignacio	Searles	64.536	isearles@itba.edu.ar
Augusto	Barthelemy Solá	10.003	abarthelemysola@itba.edu.ar

2. Dominio

Desarrollar un lenguaje que permita definir funciones recursivas primitivas (RP). Las funciones RP son de la forma $f: \mathbb{N}^k \rightarrow \mathbb{N}$. Las mismas cumplen que están definidas en base a los esquemas recursivos de tipo I y tipo II y composición a partir de las funciones iniciales (constantes, sucesor y proyección).

El lenguaje permite evaluar cualquier función RP que se haya definido. Haciendo uso de las definiciones de las mismas para evaluarlas recursivamente y obtener un valor de salida.

3. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán definir funciones.
- (II). Se proveerán maneras de componer funciones.
- (III). Se podrán definir funciones en base a esquemas recursivos (tipo I y II).
- (IV). Se podrán evaluar funciones.
- (V). Se podrán importar definiciones de funciones desde archivos externos.
- (VI). Se podrán evaluar y componer funciones binarias usandolas como operador binario. Es decir, de la forma (*expr func expr*).
- (VII). Se proveerán un conjunto de funciones básicas: constante (se puede directamente escribir el número), proyección (se realizará de forma implícita) y sucesor.
- (VIII). Se proveerá variables especiales para la entrada de valores por teclado. (*i_0, i_1, i_2, ..., i_n*).

4. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que defina una función suma.
- (II). Un programa que defina una función producto.
- (III). Un programa que evalúe la función producto utilizando la misma como operador binario (ej ($5 * 2$)).
- (IV). Un programa que defina una función resta.
- (V). Un programa que defina una función equals.
- (VI). Un programa que defina una función sumatoria acotada.
- (VII). Un programa que defina una función característica de un predicado.
- (VIII). Un programa que evalúe la función producto incluida desde un archivo externo.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa que realice una composición donde no sea correcta la cantidad de argumentos.
- (II). Un programa que realice el esquema recursivo de forma incorrecta.
- (III). Un programa que no evalúe correctamente una función.
- (IV). Un programa que utilice funciones no definidas previamente.
- (V). Un programa que utilice una función no binaria con la sintaxis de operador binario de evaluación de funciones binarias.

5. Ejemplos

Creación de una función RP definida con un ERI y la evaluamos:

```
// incluimos otro archivo con definiciones
#include prod.rp

// Crear una nueva función con un ERI:
def !(x):
-> !(0) = 1
-> !(n + 1) = (suc(n) * !(n)) // g(x, y) = *(suc(x), y)

// Evaluamos !
!(5)
```

Creación de una función RP definida con un ERII y la evaluamos:

```
// Crear una nueva función con un ERII

def +(x, y):
-> +(x, 0) = x
-> +(x, y + 1) = suc(+(x, y)) // g(x, y, z) = suc(z) (la proyección está implícita)

// Evaluamos +
(5 + 2)
```

Creación y posterior evaluación de una función RP definida como una composición:

```
// incluimos otro archivo con definiciones
#include std.rp

// Crear una nueva función:
def pow2(x):
-> pow2(x) = (x * x) // Sintaxis válida para funciones binarias
```

```
// Evaluar la función definida con entrada por teclado:  
pow2(i_0)
```